

LAA Assignment 3

Arushi Marwaha
Roll No.: MDS202512

Helper Functions Code

```
import numpy as np
import matplotlib.pyplot as plt

#----- norm function

def norm(x):
    '''
    Computes the Euclidean (L2) norm of a vector or matrix.
    Parameters:
        x : numpy array, Input vector or matrix.

    Returns:
        float, Euclidean norm of x.
    '''
    return np.sqrt(np.sum(x**2))

#----- partial pivoting function

def partial_pivoting(U,L,P,k):
    '''
    Performs partial pivoting on the k-th column during LU
    factorization.

    Parameters:
        U : numpy.ndarray, Current upper triangular matrix being
            formed during elimination.
        L : numpy.ndarray, Unit lower triangular matrix storing
            multipliers.
        P : numpy.ndarray, Permutation matrix recording row swaps.
        k : int, pivot column index.
    Returns:
        U : numpy.ndarray, Updated matrix after row pivoting.
        L : numpy.ndarray, Updated lower triangular matrix after
            row swaps.
        P : numpy.ndarray, Updated permutation matrix.
```

```

    Raises: ValueError- If all possible pivots in column k are
        zero (matrix is singular).

    '''
    n = U.shape[0]

    pivot_row = np.argmax(np.abs(U[k:n,k]))+k

    if np.abs(U[pivot_row,k]) < 1e-15:
        raise ValueError("The given matrix is singular.")

    if pivot_row!=k:
        U[[k,pivot_row]]=U[[pivot_row,k]]
        P[[k,pivot_row]]=P[[pivot_row,k]]
        if k>0:
            L[[k,pivot_row], :k] = L[[pivot_row, k], :k]

    return U,L,P

#----- LU factorization function

def lu_factorization(A):
    '''
    Performs LU factorization of a square matrix using Gaussian
        elimination
    with partial pivoting.

    Parameters:
        A : numpy.ndarray, Input square matrix.

    Returns:
        L : numpy.ndarray, Unit lower triangular matrix.
        U : numpy.ndarray, Upper triangular matrix.
        P : numpy.ndarray, Permutation matrix recording row swaps.

    Raises:
        ValueError : If the matrix is singular and pivot becomes
            zero.
    '''
    n = A.shape[0]
    U = A.copy().astype(float)
    L = np.eye(n)
    P = np.eye(n)
    for k in range(n-1):
        U, L ,P = partial_pivoting(U,L,P,k)
        for j in range(k+1,n):
            factor = U[j,k]/ U[k,k]
            L[j,k] = factor
            U[j,k:] = U[j,k:] - factor*U[k,k:]
    return L,U,P

```

```

#----- Forward substitution

def forward_substitution(L,b_dash):
    '''
    Solves the system  $Ly = b$  using forward substitution.

    Parameters:
        L : numpy.ndarray, Unit lower triangular matrix.
        b_dash : numpy.ndarray, Right-hand side vector after
            applying permutation.

    Returns:
        y : numpy.ndarray, Solution of the system  $Ly = b$ .
    '''
    n = L.shape[0]
    y = np.zeros(n)
    for i in range(n):
        y[i] = b_dash[i] - np.dot(L[i,:i],y[:i])
    return y

#----- Backward substitution

def backward_substitution(U,y):
    '''
    Solves the system  $Ux = y$  using backward substitution.

    Parameters:
        U : numpy.ndarray, Upper triangular matrix.
        y : numpy.ndarray, Right-hand side vector obtained after
            forward substitution.

    Returns:
        x : numpy.ndarray, Solution of the system  $Ux = y$ .
    '''
    n = U.shape[0]
    x = np.zeros(n)
    x[n-1] = y[n-1]/U[n-1,n-1]

    for i in range(n-2,-1,-1):
        sum_Ux = U[i, i+1:n]@x[i+1:n]
        x[i] = (y[i] - sum_Ux)/ U[i,i]
    return x

#----- Substitution solver

def lu_solver(A,b):
    '''
    Solves the system  $Ax = b$  using LU factorization with partial
        pivoting.

    Parameters:

```

```

    A : numpy.ndarray, Input square matrix.
    b : numpy.ndarray, Right-hand side vector.

Returns:
    L : numpy.ndarray, Unit lower triangular matrix.
    U : numpy.ndarray, Upper triangular matrix.
    P : numpy.ndarray, Permutation matrix.
    x : numpy.ndarray, Solution of  $Ax = b$ .
'''
L,U,P = lu_factorization(A)
y = forward_substitution(L, P@b)
x = backward_substitution(U,y)
return L,U,P,x

#----- Backward error
def backward_error(A,L,U,P):
    '''
    Calculates the backward error of the LU factorization:  $\|PA - LU\| / \|A\|$ .
    Parameters:
        A : numpy.ndarray, Original input matrix.
        L : numpy.ndarray, Unit lower triangular matrix.
        U : numpy.ndarray, Upper triangular matrix.
        P : numpy.ndarray, Permutation matrix.

    Returns:
        float: The backward error of the factorization.
    '''
    return norm(P@A - L@U)/norm(A)

#----- Residual error
def residual_error(A,x,b):
    '''
    Calculates the normalized residual error of the solution.
    Formula:  $\|Ax - b\| / \|b\|$ 
    Parameters:
        A : numpy.ndarray, Original coefficient matrix.
        x : numpy.ndarray, Computed solution vector.
        b : numpy.ndarray, Original right-hand side vector.
    Returns:
        float, The relative residual error
    '''
    return norm(A@x - b)/ norm(b)

```

```
In [12]: import numpy as np
import time
import pandas as pd
from scipy.linalg import lu, solve
import helper_function as hf
import matplotlib.pyplot as plt
```

Matrix Sizes

We test the implementation for different matrix sizes.

```
In [10]: sizes = [100, 250, 500, 1000, 2000, 3000, 4000, 5000]
```

Running Experiments

For each size:

- Generate a random matrix A and vector b
- Perform LU factorization (our implementation)
- Solve $Ax = b$
- Compare with SciPy
- Compute backward error and residual

```
In [ ]: data = []
for n in sizes:
    # generates random test matrix and right-hand side vector
    A = np.random.randn(n, n)
    b = np.random.randn(n)

    # measures time for custom lu factorization
    start_fact = time.time()
    L, U, P = hf.lu_factorization(A)
    end_fact = time.time()

    # measures time for custom forward and backward substitution
    start_solve = time.time()
    y_tmp = hf.forward_substitution(L, P @ b)
    x_computed = hf.backward_substitution(U, y_tmp)
    end_solve = time.time()

    # benchmarks scipy's built-in lu factorization
    start_sci_factor = time.time()
    P_sci, L_sci, U_sci = lu(A)
    end_sci_factor = time.time()

    # benchmarks scipy's optimized linear system solver
    start_sci_solve = time.time()
    x_sci = solve(A, b)
    end_sci_solve = time.time()

    # calculates backward and residual errors for accuracy comparison
    back_fact_error = hf.backward_error(A, L, U, P)
    sci_back_err = hf.backward_error(A, L_sci, U_sci, P_sci)
```

```

computed_residual = hf.residual_error(A, x_computed, b)
sci_residual = hf.residual_error(A, x_sci, b)

# stores all metrics in a list of dictionaries
data.append({
    'n': n,
    'Computed factorization time': end_fact - start_fact,
    'SciPy factorization time': end_sci_factor - start_sci_factor,
    'Computed solve time': end_solve - start_solve,
    'SciPy solve time': end_sci_solve - start_sci_solve,
    'Computed backward error': back_fact_error,
    'SciPy backward error': sci_back_err,
    'Computed residual': computed_residual,
    'SciPy residual': sci_residual,
})

df = pd.DataFrame(data)

```

Performance Metrics

This table compares Factorizarion time and solve time

```

In [ ]: # Table 1: Performance Metrics
df_timing = pd.DataFrame(data, columns=[
    'n',
    'Computed factorization time',
    'SciPy factorization time',
    'Computed solve time',
    'SciPy solve time'
])

print("Table 1: Computational Timing (Seconds)")
display(df_timing)

```

Table 1: Computational Timing (Seconds)

	n	Computed factorization time	SciPy factorization time	Computed solve time	SciPy solve time
0	100	0.037067	0.068885	0.008326	0.008223
1	250	0.108022	0.023788	0.003667	0.003253
2	500	0.522949	0.009370	0.003372	0.016656
3	1000	2.168139	0.025143	0.004576	0.022139
4	2000	13.434562	0.121923	0.015911	0.157801
5	3000	33.660534	0.395357	0.030201	0.325794
6	4000	71.910487	0.789965	0.044179	0.718456
7	5000	293.842843	2.042177	0.184828	2.194631

Accuracy Metrics

This table compares backward error of factorization time and residual calculation time

```
In [ ]: # Table 2: Accuracy Metrics
df_accuracy = pd.DataFrame(data, columns=[
    'n',
    'Computed backward error of factorization',
    'SciPy backward error of factorization',
    'Computed residual',
    'SciPy Residual'
])

print("Table 2: Numerical Error Analysis")
display(df_accuracy)
```

Table 2: Numerical Error Analysis

	n	Computed backward error of factorization	SciPy backward error of factorization	Computed residual	SciPy Residual
0	100	8.081510e-16	1.402056	9.047910e-15	8.856129e-15
1	250	1.654079e-15	1.401645	2.613899e-14	4.373352e-14
2	500	3.501409e-15	1.412072	5.701427e-13	1.174138e-12
3	1000	6.604177e-15	1.414152	2.240109e-13	5.218664e-13
4	2000	1.279754e-14	1.413549	2.022078e-12	4.336333e-12
5	3000	1.874646e-14	1.413751	2.269428e-11	6.690753e-11
6	4000	2.442078e-14	1.414022	6.005018e-12	1.765544e-11
7	5000	3.036289e-14	1.414095	1.418053e-11	4.200542e-11

Plots

Plot 1: Factorization Time

Plot 2: Solving Time (Forward + Backward Substitution)

```
In [ ]: # Creates a figure with two subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# Plot 1: Factorization Time
ax1.loglog(df['n'], df['Computed factorization time'], 'o-', label='Custom LU')
ax1.loglog(df['n'], df['SciPy factorization time'], 's-', label='SciPy LU')
ax1.set_title('Factorization Time Comparison ( $n^3$ )')
ax1.set_xlabel('Matrix Size  $n$ ')
ax1.set_ylabel('Time (seconds)')
ax1.grid(True, which="both", ls="-", alpha=0.5)
ax1.legend()
```

```

# Plot 2: Solving Time (Forward + Backward Substitution)
ax2.loglog(df['n'], df['Computed solve time'], 'o-', label='Custom Solve')
ax2.loglog(df['n'], df['SciPy solve time'], 's-', label='SciPy Solve')
ax2.set_title('Solving Time Comparison ( $O(n^2)$ )')
ax2.set_xlabel('Matrix Size  $n$ ')
ax2.set_ylabel('Time (seconds)')
ax2.grid(True, which="both", ls="-", alpha=0.5)
ax2.legend()

plt.tight_layout()
plt.savefig('performance_comparison.png') # Saves the plot to a file
plt.show()

```

